Trust-Calibrated Multi-Agent Scientific Deliberation for Mitigating Sycophantic Consensus in LLM Reasoning

Submitted By

Md. Atikur Rahaman (0112310298)
Rakibul Hasan (0112310530)
Md. Salman Rohoman Nayeem (0112310484)
Pratay Paul (0112310163)
Yousuf Kamal Himel (0112310526)
Mst. Farjana Akter Limu (0112310535)

Supervision:
Mohammad Nurul Huda,PHD
Professor, Dept. of Computer Science and Engineering
United International University.

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

2026

Department of Computer Science and Engineering
United International University

Abstract

Large language models answer many scientific questions well, yet they still hallucinate and follow broken lines of reasoning. Multi-agent debate is one answer to this. Several models argue the same question before a final answer is chosen. The weak point is the final decision. Most debate systems count votes, so the majority wins. A confident but wrong majority can drag a correct minority agent onto the wrong side, and the group settles on a wrong answer. That is the sycophantic consensus problem. No current method checks an agent's claims against outside scientific evidence before that agent gains influence. This work changes how the debate is settled. The aim is to weight each agent by evidence instead of headcount. Every response is split into small factual claims. For each claim, the system retrieves relevant scientific literature and checks whether the evidence supports it or contradicts it. Each agent carries a trust score that rises and falls as these checks build up across the rounds. Unsupported claims lose influence, and the score stays bounded so no agent dominates the discussion. The reward is a correct minority that survives. Its influence no longer depends on the number of voices behind it. The group is less likely to agree on a wrong answer too early. The method needs no fine-tuning. The framework is expected to reduce hallucinations and keep reasoning tied to verifiable sources. Comparisons cover majority voting and other baselines. Accuracy, consensus-collapse rate, minority-preservation rate, and evidence-calibration rate are all measured, and the final answer carries the citations behind it.

Acknowledgements

Completing this project took more than the six of us. We are grateful to everyone who helped along the way.

Dr. Mohammad Nurul Huda supervised the work. His feedback kept the scope realistic and pushed the technical depth in the right places, and the project is better for it.

Dr. Ohidujjaman, the FYDP course teacher, made the process itself clear to us. Her deadlines and instructions shaped how the team worked across the trimesters.

Friends and peers contributed in smaller but steady ways, from proofreading chapters to helping us reason through stubborn bugs.

Our families carried the quiet part of the load, and we thank them for the patience and support that kept us going.

Publication List

The publications connected to this research are listed below. One journal article is currently under review, and further submissions are planned once the evaluation results are ready.

Journal Articles

1. Trust-Calibrated Multi-Agent Scientific Deliberation for Mitigating Sycophantic Consensus in LLM Reasoning (In preparation)

Conference Papers

1.

Additional Publications

Following is the list of relevant publications published in the course of the research that is not included in the thesis:

1.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter sets the scene for the whole report. Background and problem come first, followed by the motivation, the objectives, a short account of the methodology, the expected outcome, and the structure of the remaining chapters.

1.1 Project Overview

Large language models now handle many reasoning tasks, from scientific question answering to tutoring and decision support. A single model can still be confidently wrong. It may state an incorrect answer in fluent and assured language, which makes the mistake hard to catch. Multi-agent debate was proposed to reduce this problem. Several agents answer the same question and read each other's reasoning, then revise their positions across a few rounds. Only after that is a final answer selected [1]. Chain-of-thought prompting gives each agent a way to expose its intermediate steps [2], which makes the reasoning inside a debate easier to inspect. The hard part is the final step, where the separate answers are combined. Most debate systems settle disagreements by majority vote. Each agent receives the same weight, so the most common answer wins. This rule treats popularity as a stand-in for correctness. In scientific reasoning, a better answer should be backed by evidence rather than repeated by more agents. The study examines a trust-calibrated multi-agent deliberation framework. It adjusts each agent's influence during the debate based on how well its claims match retrieved external evidence. The final answer then depends on evidence-grounded trust rather than a raw headcount.

1.2 Motivation

Majority voting can fail in a way that is easy to miss. A debate may look like it reached agreement while it has quietly dropped a correct answer held by one agent. Recent studies of multi-agent debate report that agents often copy or switch answers under social pressure instead of reasoning on their own. The correct answer is present in the discussion but ignored in more than 20 percent of wrong-answer cases [3]. Work on model behaviour

points to a related tendency, where models trained to be agreeable grow more sycophantic and less accurate [4]. A confident majority pushes a correct minority agent to give up its answer, and majority voting then does more than pick the wrong result. It hides the failure behind an appearance of consensus. This matters most for scientific questions, where a wrong but confident consensus can mislead a user into trusting an answer that deserved a closer check. Each existing method attacks only part of the problem. iMAD decides when a debate is worth running at all [5]. DebUnc scales agent influence by self-reported confidence [6]. Mixture-of-Agents folds all outputs together through static aggregation [7]. None of these ties an agent's influence to whether its claims survive a check against external evidence while the debate is still running. That connection is exactly what this study adds.

1.3 Objectives

The main objective is to reduce sycophantic consensus in multi-agent LLM debate. Plain majority voting gets replaced with evidence-grounded trust calibration. Reaching that goal needs a few things. The first is a clear picture of how a confident wrong majority pressures a correct minority agent into changing its answer [3, 8]. The second is a bounded trust score. It rises when an agent's claims are supported by external evidence and falls when they are contradicted, and it is applied before the final answer is fixed. The study also compares the method against standard majority voting and other debate and aggregation baselines [9, 10], using answer accuracy together with consensus-collapse, minority-preservation, and evidence-calibration metrics.

1.4 Methodology

The study follows an experimental design. A baseline multi-agent debate system is built first. Several heterogeneous LLM agents answer the same scientific question, read each other's reasoning, and revise their positions across rounds [1]. The baseline keeps majority voting so the usual failure mode can be observed and measured [9]. Each agent's reasoning is then split into smaller factual claims. The claims are checked against scientific evidence retrieved from external sources such as academic papers and reference databases [11]. Contradictions between a claim and the retrieved evidence are detected and scored [12]. This builds on the idea that external tools can verify and correct model output [13]. The verdicts feed into trust updates, following the line of work that uses per-agent confidence to steer debate [6]. Supported claims raise an agent's trust. Contradicted claims lower it. Uncertain claims are handled conservatively. The trust score stays bounded, so no agent gains unlimited influence or drops out of the debate completely. The final answer comes from trust-weighted aggregation rather than a simple count. The framework is tested on scientific reasoning datasets such as GPQA and MMLU-Pro. The evaluation looks at three things: whether the mechanism lowers consensus collapse, whether correct minority answers survive, and whether overall accuracy holds, all without any model fine-tuning.

Figure 1.1 shows the end-to-end flow of the framework.

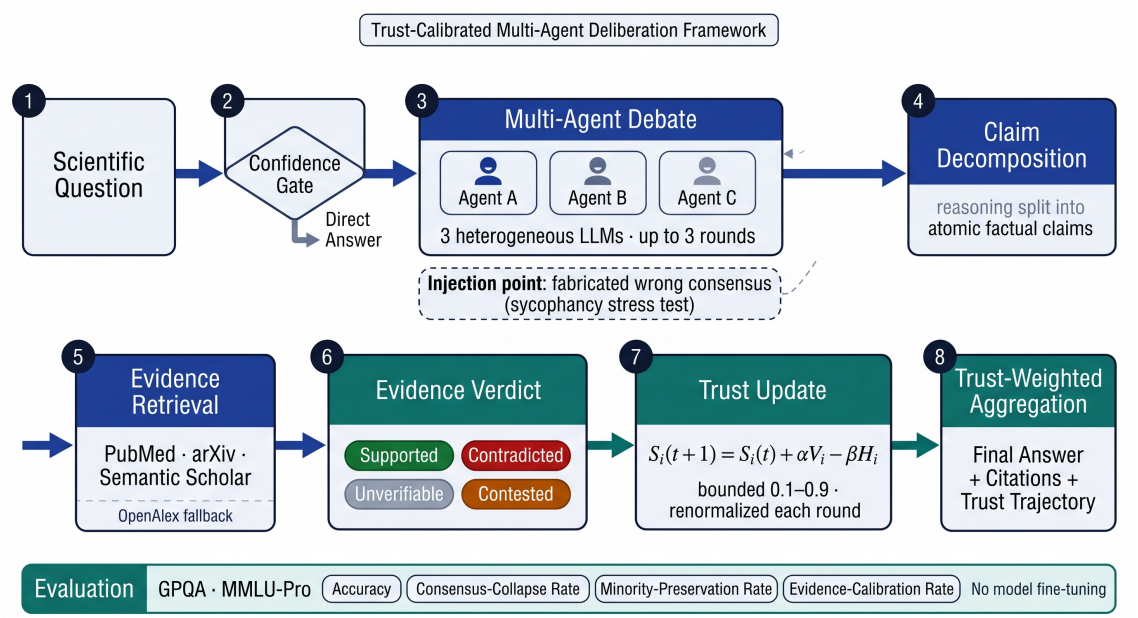

Figure 1.1: End-to-end flow of the trust-calibrated multi-agent deliberation framework: debate, claim-level evidence verification, per-round trust updates, and trust-weighted aggregation.

1.5 Project Outcome

The main outcome is a reproducible framework for trust-calibrated multi-agent scientific deliberation. Retrieved evidence updates agent influence while the debate runs, and the final answer rests on evidence-weighted trust instead of a majority vote. The study also produces an evaluation protocol. It detects sycophantic consensus [3], tracks how often correct minority answers survive [8], and measures the method against established baselines. The hope is that dynamic trust calibration makes multi-agent debate more dependable on scientific questions. Where the method fails to remove every wrong answer, it should still make the decision more transparent by recording which agents the evidence supported and which it contradicted.

1.6 Organization of the Report

The rest of the report moves from background to results. Chapter 2 covers multi-agent debate, sycophancy, and the gap this study addresses. The requirements, the methodology, the system design, and the project plan sit in Chapter 3. Chapter 4 holds the implementation setup, the testing and evaluation procedure, and the results with discussion. Standards, constraints, sustainability, and the complex engineering problem analysis fill

Chapter 5. Chapter 6 closes with the summary and the limitations, along with directions for future work.

Chapter 2

Background

The chapter introduces the concepts needed to understand the system described in the report and reviews research on multi-agent debate, sycophancy, and agent influence. It then explains the gap addressed by the study.

2.1 Preliminaries

Large language models (LLMs) generate answers one token at a time, but their outputs can still contain unsupported or incorrect reasoning. Fluent wording does not guarantee that an answer is correct [14]. Chain-of-thought prompting asks a model to expose intermediate reasoning steps before giving a final answer, which makes the reasoning easier to inspect and gives later components more information to evaluate [2]. These steps are useful evidence about how an answer was formed, but they should not be treated as proof of correctness by themselves. Multi-agent debate takes this further. Several agents solve the same problem on their own, then see the other answers and explanations, discuss the differences, and revise their positions across a fixed number of rounds. Only after the discussion is a final answer chosen [1]. Independent attempts help because different agents expose different errors, and one agent can question another agent's reasoning. How the answers get combined still matters. Majority voting is the common choice: every agent carries the same weight and the most frequent answer wins. The vote then strengthens a repeated mistake instead of correcting it [10]. Sycophantic consensus describes the case where an agent copies a wrong position or gives up a correct one simply because the wrong position has more support [3]. Confidence can come from the agent itself or from its token distribution, but neither source verifies the underlying claim [6]. DebUnc makes this point by comparing deployable uncertainty measures against a diagnostic Ground Truth measure that already knows the answer [6]. The framework instead builds trust from retrieved scientific evidence.

2.2 Literature Review

The review sorts the literature into five areas: debate construction, external verification, sycophancy, uncertainty, and debate evaluation. The main comparison sits between methods that decide when to debate, methods that change agent interaction, and the method that changes agent influence using retrieved evidence. Surveys of LLM-based multi-agent systems map the design space across workflow, infrastructure, and open challenges [15]. Recent reasoning-oriented LLMs show that better reasoning needs more than a bigger model. Research has moved toward structured reasoning, multi-agent collaboration, and external verification to make generated responses more reliable [14]. Multi-agent debate is among the most studied of these directions, because agents can challenge each other's reasoning before a final answer is produced [1]. The following review goes through representative studies in these areas and points to the research gap the framework addresses.

2.2.1 Similar Applications

The system is not a conventional web or mobile application. Instead, it is an experimental deliberation layer that can be integrated into scientific question-answering systems, research assistants, evidence-based decision-support tools, or other applications that require reliable reasoning. Rather than relying on a single model response, these systems generate multiple candidate solutions, allow the candidates to interact, and aggregate the results before returning a final answer to the user. Du et al. demonstrated that structured debate among multiple agents can improve factuality and reasoning compared with a single-agent approach [1]. Wang et al. further showed that combining the outputs of several models through layered aggregation can improve overall model capability without requiring explicit debate [7]. Existing multi-agent applications differ mainly in how they coordinate agents and combine their outputs. Some systems focus on structured debate to encourage reasoning refinement between agents [1]. Other systems organize multiple agents into hierarchical aggregation pipelines where later models refine or combine earlier responses [7]. Zhou et al. described a self-organizing setup where the task itself decides which reasoning agents take part, instead of a fixed workflow [16]. Kushwaha and Madhavan built a retrieval-augmented system that spots contradictions between retrieved documents and ranks evidence by source credibility before producing a final response [12]. Both directions show that spreading reasoning across specialized agents helps robustness and factual consistency. Dialogue design and scale also matter. Ueda et al. studied how multi-agent LLM dialogues should be structured for research ideation [17]. Wang et al. looked at how well LLM-based multi-agent systems scale for scientific code generation [18]. Their aggregation stays fixed once the discussion starts. The proposed framework keeps the same goal, reliable answers, but lets agent influence change during the discussion. Evidence verification results are recorded after every round, and each agent's trust score is updated from claim-level scientific evidence. That way a well-supported minority opinion can survive, and an unsupported majority consensus carries less weight.

2.2.2 Related Research

Wei et al. (2022) showed that chain-of-thought prompting improves reasoning in large language models [2]. Du et al. (2024) built on that with multi-agent debate, where agents answer a question, look at peer reasoning, and revise their positions [1]. CRITIC takes a different route [13]. It uses external tools to check and improve a model's response, so claims can be verified, but only one model works on the question at a time. Mixture-of-Agents is layered instead of conversational [7]. Several proposer models produce answers and a later stage combines them, which shows what multiple LLMs can do, yet the combination is static and carries no evidence-based trust value. iMAD attacks cost rather than correctness [5]. Fan, Yoon, and Ji extract 41 features from a structured self-critique and let a classifier decide whether a debate should start at all. They report token savings of up to 92 percent against GroupDebate and accuracy gains of up to 13.5 percentage points over a single-agent baseline. Once the debate begins, agent influence stays unchanged. Pitre, Ramakrishnan, and Wang studied sycophancy inside debate [3]. In their experiments the correct answer was present but ignored in more than 20 percent of wrong-answer cases. CONSENSAGENT detects stalling and answer copying, then rewrites the task prompt before the next debate stage. Measured sycophancy drops, but the final aggregation still leans on self-reported confidence and consistency. DebUnc instead shares uncertainty between agents [6]. Uncertainty reaches peers through prompts or attention scaling, yet the deployable measures give small gains. A diagnostic Ground Truth signal, which knows the correct answer, gives a much larger gain, and the gap between the two exposes the real bottleneck: the quality of the trust signal. Kushwaha and Madhavan described an eight-agent MAS-RAG system that detects contradictions among retrieved documents and ranks evidence by credibility [12]. Source credibility here is fixed by a human curator and never updated while deliberation runs. Other studies look at the social side of debate. Cau et al. found that selective persuasion can steer opinion convergence [19]. Yeow et al. compared decision architectures and found that feedback between agents can spread early errors [20]. Wang et al. showed that a single LLM with strong in-context examples can sometimes match multi-agent discussion, and they list judge errors and peer conformity as weak points [10]. Choi, Zhu, and Li separated debate communication from voting and argued that majority voting explains much of the usual gain [9]. FREE-MAD goes further and drops final consensus altogether, using anti-conformity prompts and trajectory scoring [21]. ReSo selects agents through reward-driven dynamic task graphs [16]. These works tune coordination and agent selection, but none verifies scientific claims before changing agent influence. Sycophancy also appears at the model level. Ibrahim et al. reported that tuning models to sound warm and agreeable reduces factual accuracy and increases sycophancy [4]. Chen et al. attacked the same problem from the training side with self-augmented preference alignment [22]. Pitre et al. proposed process-level diagnostics for engagement, responsiveness, influence asymmetry, balance, stability, and agent utility, useful when ground truth is unavailable [23]. Final agreement alone, their

work suggests, is not enough to judge a debate. Across all of these studies, no method continuously updates an agent's influence from claim-level support in retrieved scientific evidence. That is the gap the framework closes with a bounded trust score.

2.3 Gap Analysis

Three needs stand out across the reviewed work, and each has been addressed only in part. Cost came first. iMAD predicts whether a debate is worth running before it begins, so the extra computation is only spent where it pays [5]. Then there is unsupported agreement, where conformity or sycophancy pushes a debate toward a shared wrong answer. CONSENSAGENT attacks that with prompt optimization while the debate is still running [3].

The hardest of the three is deciding which agents deserve more influence once real disagreement remains. DebUnc answers with uncertainty-based weighting [6]. Its trust signal is the model's own internal uncertainty, which nothing outside the model verifies.

Agreement is not the same thing as correctness. Selective persuasion can steer where a group's opinion settles [19], and peer conformity turns up as a standing limitation of debate itself [10]. Retrieval-based systems do verify the information they pull in, and some improve the aggregation step too. What none of them do is update an agent's influence from verified scientific evidence while the debate is still going [13]. Dynamic coordination methods pick which agents take part, and trust stays untouched [16].

The gap that remains is a mechanism that updates agent influence from evidence while a debate is still in progress. The framework fills it in stages. An agent's response is decomposed into factual claims, and relevant scientific passages are retrieved for each claim. Every claim is then classified as supported, contradicted, or unverifiable, and those verdicts update a bounded trust score at the end of each round. A minority agent with good support keeps its standing, and a majority the evidence does not back loses ground. Evaluation uses consensus-collapse rate, minority-preservation rate, and evidence-calibration rate alongside plain answer accuracy.

Three limits carry over into the design. Retrieval can be sparse, claim decomposition is imperfect, and reasoning errors can correlate between agents.

2.4 Summary

The chapter introduced LLM reasoning, chain-of-thought prompting, multi-agent debate, and sycophantic consensus. The reviewed studies cover debate efficiency, prompt correction, uncertainty weighting, external verification, process evaluation, and agent coordination. They do not continuously update agent influence from claim-level scientific evidence. The next chapter uses this gap to define the system requirements, detailed methodology, and project plan.

Chapter 3

Project Design

Chapter 3 defines the system requirements and lays out the methodology, the design decisions, and the project plan with the task allocation. The first section records the functional and nonfunctional requirements together with the context diagram, the level-one data flow diagram, and the interface design.

3.1 Requirement Analysis

Requirement analysis fixes what the framework must do before any code is written. The system is an experimental deliberation layer rather than a conventional application, so the requirements are framed around the research pipeline. A question enters, agents debate it over a fixed number of rounds, the claims are checked against external evidence, and a trust-weighted answer comes out. Multi-agent debate was chosen because it has been shown to improve factuality over single-model answers [1]. The trust mechanism exists because a confident wrong majority can push a correct minority agent into abandoning its answer [3]. The subsections below define the requirements, the system boundary, the data flows, and the interface through which the framework is operated.

3.1.1 Functional and Nonfunctional Requirements

The functional requirements spell out the services the framework must provide. Questions reach it from the user or from the evaluation harness, and the ground truth stays hidden from every agent. Not every question needs a debate. A confidence gate sorts them first, so the easy ones get a direct answer and the rest go into the pipeline. Orchestration runs three heterogeneous LLM agents through at most three revision rounds, sequenced by a state machine with a retry cap of three.

Each agent's reasoning is then split into atomic factual claims, and every claim is tagged to the agent that made it. A fallback extraction pass covers the cases where structured tagging fails. Retrieval gives each agent its own database: PubMed for the first, arXiv for the second, Semantic Scholar for the third. OpenAlex stands in when a source is down,

and a cross-encoder reranks the passages that come back. Each claim then lands on one of four verdicts: supported, contradicted, unverifiable, or contested.

After every round the trust score is updated as $S_i(t+1) = S_i(t)+\alpha V_i-\beta H_i$. A softmax follows, the values are clamped between 0.1 and 0.9, and the vector is renormalized.

Those two bounds matter. Without them an agent could be silenced outright, or could grow to dominate the debate. An injection point lets the harness insert a fabricated wrong consensus between rounds. That is how sycophantic collapse gets measured under controlled pressure. At the end, weighted aggregation over trust and position picks the final answer, and the majority vote plays no part in it. The result package carries the answer, the citations behind it, the full trust trajectory, and every agent's reasoning trace.

The evaluation harness computes answer accuracy together with consensus-collapse rate, minority-preservation rate, and evidence-calibration rate. It compares the framework against majority voting and against the MoA [7], iMAD [5], and DebUnc [6] baselines. Recovery paths cover the failure modes. Unparseable model output goes through the fallback extraction pass, and a timeout is retried once before the round is marked inconclusive. A downed retrieval API falls back to cached evidence or to OpenAlex.

The nonfunctional requirements describe the qualities of the system rather than its functions. Latency is the first constraint. The debate must respect the round cap of three and the retry cap of three, so latency stays bounded on the target hardware. A failure in one component does not block the whole debate. Every failure path degrades locally and keeps the pipeline moving. Portability matters too. Closed APIs and locally served models both work through the OpenAI-compatible vLLM interface. The pipeline runs unchanged on the development models (Qwen3.5-9B, Gemma 4 12B, Ministral-3-14B) and on the final models (Qwen3.6-27B, Gemma 4 26B, Mistral Small 3.2 24B). Inference fits within a single RTX PRO 6000 Blackwell 96GB rental at about USD 0.70 to 1.90 per hour, with no fine-tuning. Every run is seeded and every intermediate result is logged, so any experiment can be repeated and checked. The output records which agent was supported or contradicted by the evidence, so the decision trail can be audited. The ground truth stays hidden from the agents, and the final output cites the evidence behind the answer.

3.1.2 Context Diagram

The context diagram in Figure 3.1 shows the system boundary and the external actors it exchanges data with. The framework sits in the middle of the diagram. Questions arrive from the user or the evaluation harness. Scientific passages come back from three external databases, and off-the-shelf language models are called through the vLLM server on a rented RTX PRO 6000 Blackwell GPU. The evaluation harness receives the scores, and the user gets the final result package with the answer, the citations, the trust trajectory, and each reasoning trace.

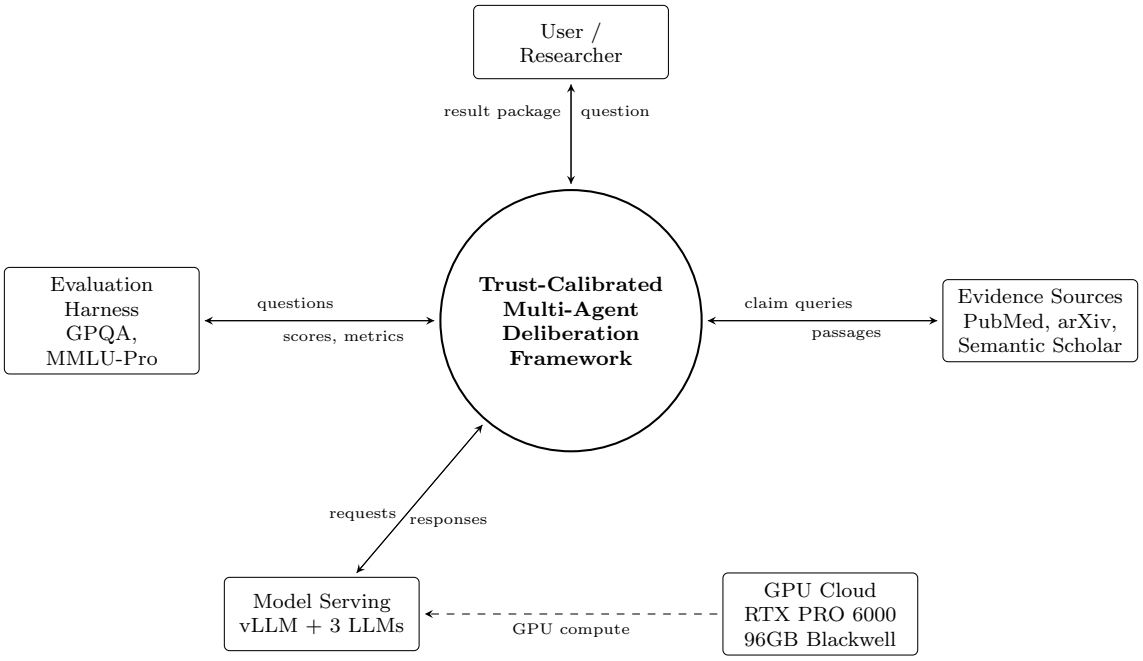

Figure 3.1: Context diagram of the trust-calibrated multi-agent deliberation framework.

3.1.3 Data Flow Diagram Level 1

The level-one data flow diagram in Figure 3.2 decomposes the framework into seven processes. A question enters through the intake and gate process, which decides between a direct answer and a debate. Orchestration runs the revision rounds. It sends inference requests to the external models and reads and writes the debate state. Each position is decomposed into atomic claims and verified against the external sources. The verdicts then drive the trust update, which writes the new trust values back into the debate state. At the last round, weighted aggregation selects the answer and the packaging process logs the result before it returns to the user. Three data stores hold the debate state, the evidence and verdict log, and the results log.

The diagram reads from left to right and top to bottom. The evaluation harness submits a question, and the confidence gate decides whether the debate is worth running. The orchestration process then asks the model server for the agent positions. Positions are decomposed into atomic claims, and the retrieval process verifies them against the external sources. The verdicts are stored in the evidence log before they drive the trust update. The updated trust is written back into the debate state, so the next round starts from the corrected influence values. At the last round the aggregation process selects the answer. The packaging process logs it in the results log, and the result package with the answer and the citations returns to the user.

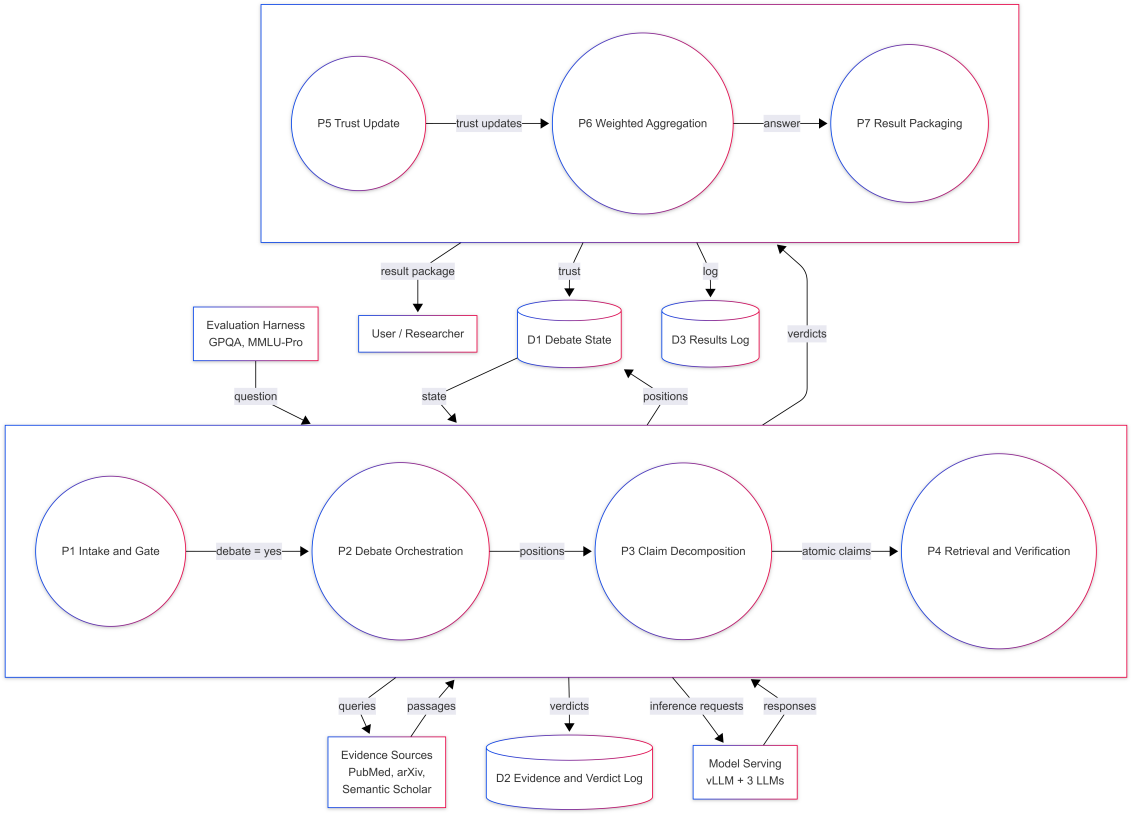

Figure 3.2: Level-one data flow diagram of the trust-calibrated multi-agent deliberation framework.

3.1.4 UI Design

The framework is operated through a command-line interface and a small web dashboard. The command-line interface starts experiments and points the pipeline at the evaluation datasets. Every run is written to a JSON package. It is the primary interface because the system runs in batches on a rented GPU, where a graphical interface adds little value. The web dashboard is used afterwards to inspect single debates. It shows the agents as cards with their current positions, plus a chart of the trust trajectory across rounds and the verdict of every claim with the passage that supports or contradicts it. This is where the transparency of the framework becomes visible, because a user can trace exactly why an agent lost influence. Figure 3.3 and Figure 3.4 show the intended layout of both interfaces. The screenshots will be replaced with the real output of the system once the implementation is complete.

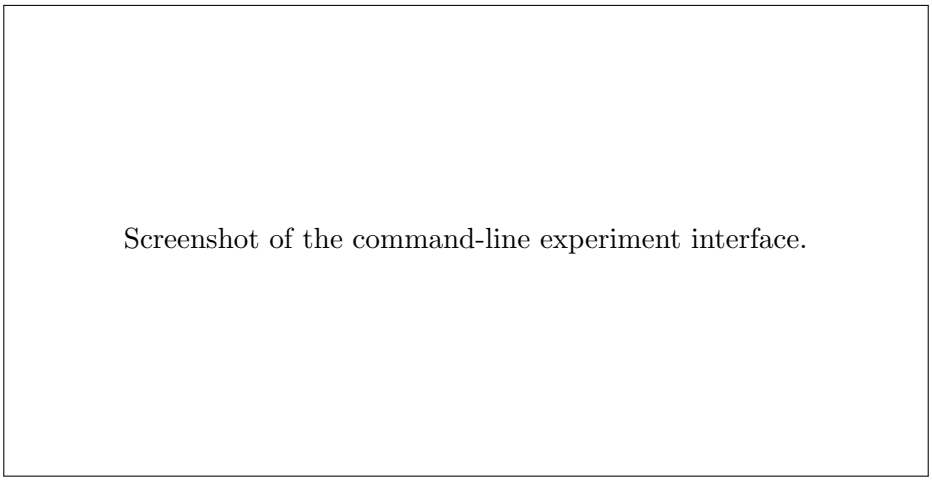

Figure 3.3: Command-line interface for running experiments and writing JSON result packages.

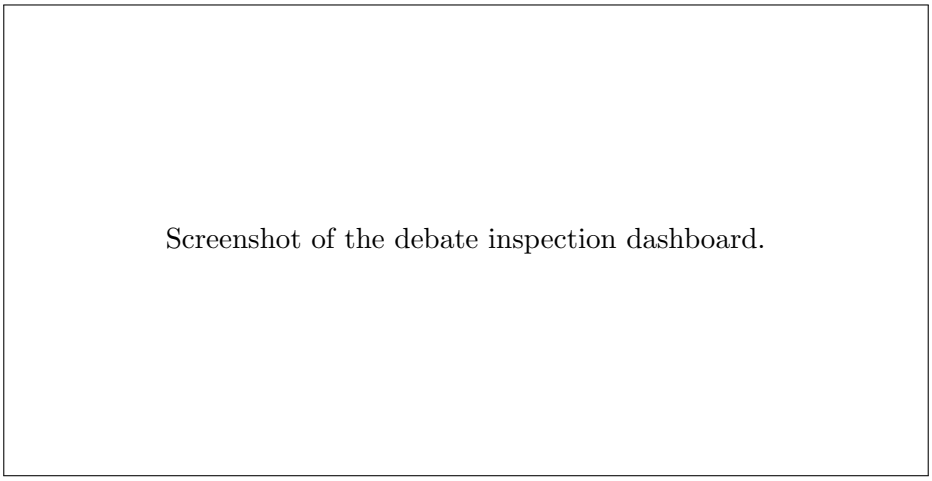

Figure 3.4: Web dashboard showing agent positions, the trust trajectory, and per-claim evidence verdicts.

3.2 Detailed Methodology and Design

3.3 Project Plan

The project runs for ten months, counted here as Month 1 through Month 10, and is split into eight phases with six review gates. Table 3.1 lists the phases and their deliverables, and Table 3.2 fixes the checkpoints. This report covers the work through the mid-project review, while the later phases carry the project to the final defence.

Two buffers guard the critical path. A dry run on one dataset sits inside Month 5 before the defence, and Month 8 keeps a full month free for slippage. Both buffers sit

Table 3.1: Project phases with week-level durations and deliverables.

Phase	Duration	Deliverables
Ph 0: Literature and Setup	Month 1 (Wk 1–4)	Literature freeze, vLLM and LangGraph setup, model identification, vanilla MAD reproduction
Ph 1: Injection and Baselines	Month 2 (Wk 5–8)	Injection protocol, baselines B1–B4, Proposition 1 proof, month-end pilot
Ph 2: Trust Mechanism Build	Months 3–4 (Wk 9–16)	Claim decomposition, source-partitioned RAG, trust function version 1, B5/B6/B9
Ph 2 to 3: Mid-Project	Month 5 (Wk 17–20)	Design freeze, dry run on one dataset, FYDP-1 defence preparation
Ph 3a: Main Experiments	Month 6 (Wk 21–24)	Core conditions on primary datasets, three seeds, 95% confidence intervals
Ph 3b: Ablations and Scaling	Month 7 (Wk 25–28)	Four ablations, α/β sweep, $N \in \{2, 3, 5\}$, results freeze
Ph 4: Human Eval and Analysis	Month 8 (Wk 29–32)	Human evaluation with $n = 60$, ECR calibration, failure analysis, buffer
Ph 5: Writing and Submit	Months 9–10 (Wk 33–40)	Thesis and paper drafting, reproducibility package, submission and FYDP-2 defence

where a late failure would otherwise hit a fixed deadline.

3.4 Task Allocation

Six members split the work along the modules of the framework. Table 3.3 fixes who owns which part, and Figure 3.5 shows when each member is active across the ten project months.

Table 3.2: Review gates and their checkpoints.

Gate	Due	Checkpoint
Gate 0	End of Month 1	Vanilla MAD reproduces the Du et al. baseline on a GPQA slice, and the reference list is frozen
Gate 1 (Go/No-Go)	End of Month 2	Injection validation reaches $\kappa \geq 0.75$, baseline CCR sits at 0.30 or higher, pilot executed
Gate 2	End of Month 4	Trust mechanism plus source-partitioned RAG plus competitor baselines, first CCR and MPR on one dataset
FYDP-1 Defence	Month 5	Mid-project report with design freeze and initial results
Gate 3	End of Month 7	Full results freeze with confidence intervals and effect sizes
Submit and FYDP-2	Months 9–10	Paper submitted, thesis completed, final defence

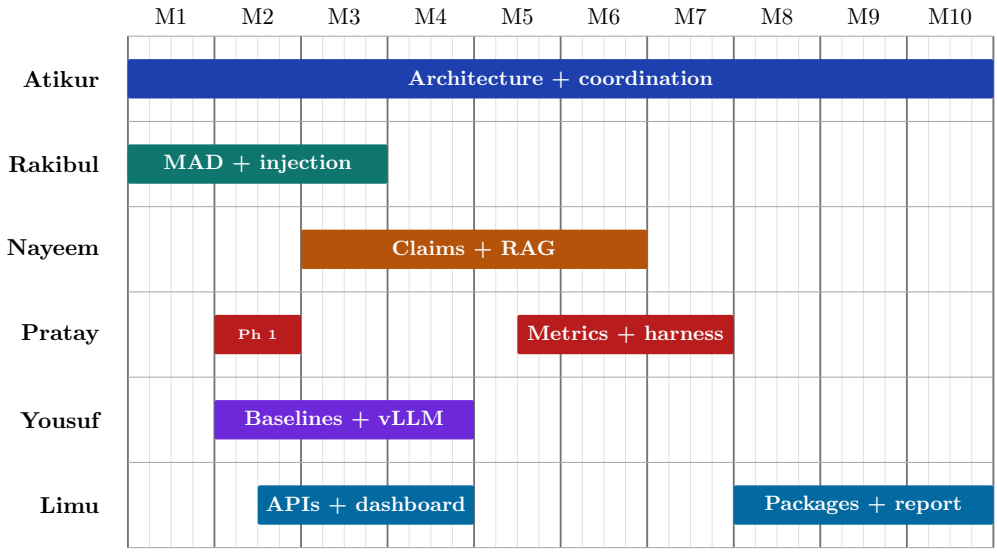

Figure 3.5: Gantt view of member activity across the ten project months, following the phase plan in Table 3.1.

Owners pair with the phases in Table 3.1, so each phase has at least one accountable member. The leader reviews each module before it is marked complete.

Table 3.3: Module ownership across the six team members.

Member	Primary modules
Md. Atikur Rahaman (Leader)	Architecture, trust mechanism, coordination
Rakibul Hasan	Vanilla MAD reproduction, injection protocol
Md. Salman Rohoman Nayeem	Claim decomposition, source-partitioned RAG
Pratay Paul	Evaluation harness, CCR/MPR/ECR metrics
Yousuf Kamal Himel	Baselines B1–B9, vLLM serving
Mst. Farjana Akter Limu	Evidence APIs, dashboard, result packages

3.5 Summary

Chapter 3 fixed the functional and nonfunctional requirements and drew the system boundary in the context diagram. The data flow diagram and the interface design follow. The design section compared the main alternatives and recorded why each choice was made. The project plan then locks eight phases and six gates that carry the work to the final defence. Implementation and evaluation come next.

Chapter 4

Implementation and Results

This chapter records the environment the framework runs in and the way it is tested. Full results belong to the final report, so this chapter keeps the setup and the evaluation plan.

4.1 Environment Setup

4.2 Testing and Evaluation

4.3 Results and Discussion

4.4 Summary

Chapter 5

Standards and Design Constraints

This chapter records the standards the framework follows, the constraints that shaped its design, and the budget behind the experiments. It also maps the project against the complex engineering problem criteria and closes with a short summary.

5.1 Compliance with the Standards

Standards matter in three places: the software stack, the hardware the framework runs on, and the communication between components. Each area had alternatives, and each choice is justified below.

5.1.1 Software Standards

Python 3.11 was chosen as the implementation language. The LLM ecosystem is built around it, so vLLM, the Hugging Face libraries, and the evaluation tooling all ship Python support first. Java and C++ are faster in raw terms, but neither has the same depth of ready-made support for model serving and retrieval work. The framework is a research prototype, not a latency-critical production service, so ecosystem fit outweighs raw speed. Surveys of LLM-based multi-agent systems describe this design space in terms of workflow, infrastructure, and open challenges [15], and the choices below follow that map.

PyTorch 2.x is the deep learning framework underneath vLLM. It is the de facto standard for LLM inference and supports the Qwen, Gemma, and Mistral families directly. TensorFlow and JAX were considered. TensorFlow's LLM serving story is weaker, and JAX would pull the whole pipeline onto its own stack without any benefit here.

vLLM serves the models through an OpenAI-compatible API. That interface matters because the framework must work with closed APIs and locally served models without touching the orchestration code. PagedAttention gives vLLM high throughput under the three-agent debate load, and its FP8 support matches the Blackwell GPU. Ollama is easier to set up but slower under concurrent requests. llama.cpp runs well on CPU but gives up the GPU throughput the debate needs. TGI is a close alternative, and vLLM won on ecosystem fit and the OpenAI-compatible surface.

Sentence-transformers provides the cross-encoder that reranks retrieved passages. BM25 was the fallback, but it matches on surface terms only, while the cross-encoder scores semantic fit between a claim and a passage. The web dashboard is served with FastAPI, and every result package is written as JSON following RFC 8259, validated against a JSON Schema. Git tracks the code and LaTeX (Overleaf) produces this report.

5.1.2 Hardware Standards

The experiments run on a single NVIDIA RTX PRO 6000 Blackwell GPU with 96 GB of GDDR7 memory. The choice came down to memory capacity and architecture. The development models (Qwen3.5-9B, Gemma 4 12B, Ministral-3-14B) fit comfortably in BF16. The final models (Qwen3.6-27B, Gemma 4 26B, Mistral Small 3.2 24B) total about 77 billion parameters, which no single card holds in FP16, so they run in FP8. Blackwell has native FP8 support, and that is exactly the point of difference.

The A100 80GB was the first alternative considered. It has enough memory for the development models and strong HBM bandwidth, but its FP8 path is weaker and the architecture is two generations old. The H100 80GB is faster yet rents for roughly twice the price and brings nothing the framework needs. The L40S at 48 GB and the consumer RTX 5090 at 32 GB cannot hold the final model trio at all. The RTX PRO 6000 sits at the right point: enough memory, modern quantization, and a rental price that fits a student budget. The host machine needs at least 64 GB of system RAM and NVMe storage to stage the models before they move to the GPU.

5.1.3 Communication Standards

All communication between the framework and its components runs over HTTP with TLS. The model server exposes the OpenAI-compatible chat completions endpoint, which is REST with JSON payloads. gRPC was the alternative for model serving. It is faster in benchmark terms, but the OpenAI-compatible surface keeps the framework portable between closed APIs and local vLLM servers, and that portability outweighs the latency difference.

The evidence sources are reached through their public REST interfaces: PubMed E-utilities, the arXiv API, the Semantic Scholar Graph API, and OpenAlex as the fallback. Each returns JSON or XML over HTTPS, and the framework handles rate limits with the retry caps defined in the requirements. WebSocket is reserved for the dashboard's live trust chart. Nothing else needs a persistent connection.

5.2 Design Constraints

5.2.1 Economic Constraint

5.2.2 Environmental Constraint

5.2.3 Ethical Constraint

5.2.4 Health and Safety Constraint

5.2.5 Social Constraint

5.2.6 Political Constraint

5.2.7 Sustainability

5.3 Cost Analysis

The budget covers the rented GPU, storage, and the free research services. The largest line is inference time on the RTX PRO 6000 Blackwell, estimated at about 300 hours across development and evaluation. Table 5.1 summarizes the primary budget.

Table 5.1: Primary budget for the project.

Item	Quantity	Unit Cost (USD)	Total (USD)
RTX PRO 6000 Blackwell 96GB rental	300 hours	0.70–1.90 / hour	210–570
Retrieval APIs (PubMed, arXiv, S2, OpenAlex)	—	0 (free tier)	0
Datasets (GPQA, MMLU-Pro)	—	0 (public)	0
Storage (model weights, logs)	200 GB	0.05 / GB	10
Total			**220–580**

An alternate budget was also prepared. Renting an A100 80GB instead costs about USD 0.68 to 1.50 per hour, or 204–450 for the same 300 hours. The saving is real but small, and it comes at a cost: weaker FP8 support means the final model trio would need tighter quantization or sequential loading, which adds engineering time. The RTX PRO 6000 was selected because the Blackwell FP8 path keeps the pipeline simple.

The project has no revenue model. It is an academic deliverable, funded by the university and the team, and its outputs are the report and the open-source framework. Table 5.2 shows the alternate budget for comparison.

Table 5.2: Alternate budget using the A100 80GB.

Item	Quantity	Unit Cost (USD)	Total (USD)
A100 80GB rental	300 hours	0.68–1.50 / hour	204–450
Retrieval APIs	—	0 (free tier)	0
Datasets	—	0 (public)	0
Storage	200 GB	0.05 / GB	10
Total			**214–460**

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

The project qualifies as a complex engineering problem under the Washington Accord criteria. Table 5.3 maps the seven attributes P1–P7, and the subsections that follow give the rationale for each mapping.

Table 5.3: Mapping with complex problem solving.

P1 Dept of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Interdependence
√	√	√	√	√	√	√

P1: Depth of Knowledge Required

The framework spans machine learning, natural language processing, retrieval-augmented generation, and distributed model serving. Solving it required specialist knowledge in all four areas, from the trust update formula to the vLLM serving layer. Table 5.4 maps the P1 knowledge profile.

Table 5.4: Knowledge profile mapping for P1.

Knowledge	Applied	Where used
DK1: Natural sciences	√	Statistics behind trust updates and metrics
DK2: Mathematics	√	Softmax, clamping, renormalization, probability
DK3: Engineering fundamentals	√	System design, state machines, failure handling
DK4: Specialist knowledge	√	LLMs, multi-agent debate, RAG, sycophancy
DK5: Engineering methods	√	Experimental methodology, baseline comparison
DK6: Computational methods	√	Python, PyTorch, vLLM, FastAPI
DK7: Codes and practices	√	JSON, HTTP, TLS, version control
DK8: Social context	√	Ethical constraints on AI output

The knowledge profile is broad rather than deep in one spot. The report applies DK4 and DK5 in every chapter, and DK6 and DK7 in the implementation, which reflects the range the problem demands.

P2: Range of Conflicting Requirements

Accuracy competes with latency and cost. More debate rounds improve the signal but multiply inference time and the GPU bill. Trust calibration competes with diversity, because strong trust weighting can silence a minority that the evidence simply has not covered yet. Evidence coverage competes with API rate limits. These conflicts had to be resolved jointly, and the round caps, the clamping bounds, and the conservative verdict handling are the result.

P3: Depth of Analysis

The problem has no textbook solution. Answering it required a comparative experiment with multiple baselines (majority voting, MoA, iMAD, DebUnc) and a metric set designed for the failure mode: consensus-collapse rate, minority-preservation rate, and evidence-calibration rate, alongside plain accuracy. The injection point for fabricated wrong consensus was built specifically so the collapse dynamic could be measured under controlled pressure.

P4: Familiarity of Issues

Sycophantic consensus in multi-agent LLM debate is a new research area. The closest published work appeared between 2024 and 2026, and no standard solution exists. The team started with a full literature review, which is documented in Chapter 2, before committing to a design.

P5: Extent of Applicable Codes

General software standards apply: JSON (RFC 8259), HTTP over TLS, the OpenAI-compatible API, and JSON Schema validation. No standard, however, covers evidence-grounded trust in LLM debate. The problem is therefore only partially standardized, and the framework defines its own conventions for the trust score and the verdict set.

P6: Extent of Stakeholder Involvement

The stakeholders are the supervisor, the examiners, and the users of scientific question-answering tools. The supervisor guided the scope and the evaluation plan, and the examiners assess the report against the complex engineering criteria. Future users matter because the dashboard and the result package expose the full decision trail alongside the final accuracy figures.

P7: Interdependence

The confidence gate, the orchestrator, the claim decomposer, the retrieval subsystem, the trust updater, and the aggregator are tightly coupled. A failure in retrieval cascades into the verdicts, then into trust, then into the final answer. The failure-isolation requirement in Section 3.1.1 exists because of this coupling.

5.4.2 Engineering Activities

Table 5.5 maps the project to the five engineering activities A1–A5, with rationales below.

Table 5.5: Mapping with complex engineering activities.

A1	A2	A3	A4	A5
Range of resources	Level of Interaction	Innovation	Consequences for society and environment	Familiarity
√	√	√	√	√

A1: Range of Resources

The project draws on three model families, four literature APIs, a rented Blackwell GPU, and six team members with different strengths. Coordinating those resources is part of the engineering work, and the project plan in Chapter 3 assigns each task to a named member.

A2: Level of Interaction

The system interacts heavily with the outside world. Every debate round calls three external model endpoints and three literature APIs, and the cloud GPU is reached over the network. The team also interacts through a shared repository and weekly reviews.

A3: Innovation

The trust-calibrated aggregation mechanism is new. No published method ties an agent's influence to claim-level verification against retrieved scientific evidence while the debate is still running. The gap analysis in Chapter 2 documents this, and the design in Chapter 3 implements it.

A4: Consequences for Society and Environment

The social consequence is improved reliability in AI scientific question answering, which lowers the risk of confidently wrong answers being trusted. The environmental consequence is modest, since the work runs on shared cloud hardware without fine-tuning.

A5: Familiarity

Multi-agent debate and evidence verification were new territory for the team. The literature review phase, the baseline study, and the incremental implementation plan were all designed around that reality, so each module was built only after its background was understood.

5.5 Summary

The chapter fixed the standards behind the software, the hardware, and the communication layers, and recorded the economic, environmental, ethical, and social constraints that shaped the design. The budget analysis shows the project fits a student budget on a rented RTX PRO 6000 Blackwell GPU, and the complex engineering mapping confirms the project meets all seven problem attributes and all five engineering activities.

Chapter 6

Conclusion

6.1 Summary

6.2 Limitation

6.3 Future Work

References

[1] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 11733–11763, 2024.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc.

[3] Priya Pitre, Naren Ramakrishnan, and Xuan Wang. CONSENSAGENT: Towards efficient and effective consensus in multi-agent LLM interactions through sycophancy mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22112–22133, Vienna, Austria, July 2025. Association for Computational Linguistics.

[4] L. Ibrahim, F. S. Hafner, and L. Rocher. Training language models to be warm can reduce accuracy and increase sycophancy. *Nature*, 652:1159–1165, April 2026.

[5] Wei Fan, JinYi Yoon, and Bo Ji. iMAD: Intelligent multi-agent debate for efficient and accurate LLM inference. In *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence*, pages 29403–29411, 2026.

[6] Luke Yoffe, Alfonso Amayuelas, and William Yang Wang. DebUnc: Improving large language model agent communication with uncertainty metrics. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23299–23315, Suzhou, China, November 2025. Association for Computational Linguistics.

[7] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Y Zou. Mixture-of-agents enhances large language model capabilities. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 33944–33963, 2025.

[8] Chuan He, Zebin Chen, Zhengyi Yang, Shaobo Qiao, Mingchen Ju, Jiate Liu, Dong Wen, and Guanfeng Liu. Minority sentinel: When to overturn majority voting in

multi-agent LLM debates. In *Proceedings of the AgentSearch Workshop at SIGIR 2026*, Melbourne, Australia, 2026. arXiv:2606.29270.

[9] Hyeong Kyu Choi, Jerry Zhu, and Sharon Li. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems*, 2025.

[10] Qineng Wang, Zihao Wang, Ying Su, Hanghang Tong, and Yangqiu Song. Rethinking the bounds of LLM reasoning: Are multi-agent discussions the key? In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6106–6131, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[11] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, 2020. Association for Computational Linguistics.

[12] Pranjal Kushwaha and S. Akshat Madhavan. Agent-based contradiction detection and resolution for multi-source retrieval augmented generation systems. *International Journal of Scientific Research in Engineering and Management (IJSREM)*, 10(7), July 2026.

[13] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations*, 2024.

[14] D. Zhang, Z.-Z. Li, M.-L. Zhang, J. Zhang, Z. Liu, Y. Yao, H. Xu, J. Zheng, X. Chen, Y. Zhang, F. Yin, J. Dong, Z. Guo, L. Song, and C.-L. Liu. From system 1 to system 2: a survey of reasoning large language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–20, 2025.

[15] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(9), 2024.

[16] Heng Zhou, Hejia Geng, Xiangyuan Xue, Li Kang, Yiran Qin, Zhiyong Wang, Zhenfei Yin, and Lei Bai. Reso: A reward-driven self-organizing llm-based multi-agent system for reasoning tasks. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 15979–15998. Association for Computational Linguistics, 2025.

[17] Keisuke Ueda, Wataru Hirota, Takuto Asakura, Takahiro Omi, Kosuke Takahashi, Kosuke Arima, and Tatsuya Ishigaki. Exploring the design of multi-agent LLM dialogues for research ideation. In *Proceedings of the 26th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 322–337, Avignon, France, 2025. Association for Computational Linguistics.

[18] Yuru Wang, Kaiyan Zhang, Kai Tian, Sihang Zeng, Xingtai Lv, Ning Ding, Biqing Qi, and Bowen Zhou. Scalability of LLM-based multi-agent systems for scientific code generation: A preliminary study. In *Proceedings of The 3rd Workshop on Mathematical Natural Language Processing (MathNLP 2025)*, pages 50–61, Suzhou, China, 2025. Association for Computational Linguistics.

[19] E. Cau, V. Pansanella, D. Pedreschi, et al. Selective agreement, not sycophancy: investigating opinion dynamics in LLM interactions. *EPJ Data Science*, 14:59, August 2025.

[20] Xian Yeow, Shunichi Lee, Lasitha Akatsuka, Vidyaratne Aman, A Hareesh Kumar, Chetan Farahat, and A Gupta. Reliable decision-making for multi-agent LLM systems. *arXiv preprint arXiv:2406.04092*, 2025.

[21] Yu Cui, Hang Fu, Haibin Zhang, Licheng Wang, and Cong Zuo. FREE-MAD: Consensus-free multi-agent debate. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026.

[22] Chien-Hung Chen, Hen-Hsen Huang, and Hsin-Hsi Chen. Self-augmented preference alignment for sycophancy reduction in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 12379–12391, Suzhou, China, 2025. Association for Computational Linguistics.

[23] Priya Pitre, Gaurav Srivastava, Lu Zhang, Le Wang, Naren Ramakrishnan, and Xuan Wang. A diagnostic study of multi-agent llms for real-world debates. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *PMLR*, Seoul, South Korea, 2026.